

A.B.V.M. Agrawal Jatiya Kosh's

Ghanshyamdas Jalan College of Science & Commerce

Malad (East), Mumbai

(Affiliated to University of Mumbai)

DEPARTMENT OF INFORMATION TECHNOLOGY

Internet of Things Practical

B.Sc. Information Technology

Submitted by

Ayesha Sarguru

Seat Number: 502520019

Under the Guidance of

Prof. Gufran Qureshi

Academic Year 2026–27

October 2026



Ghanshyamdas Jalan College of Science & Commerce

Malad (East), Mumbai

(Affiliated to University of Mumbai)

Certificate by the Institute

Department of Information Technology

I hereby certify that Ms.**Ayesha Sarguru**, Roll Number 20, a student of Ghanshyamdas Jalan College of Science & Commerce, Malad (East), studying in **B.Sc. Information Technology**, has successfully completed the practical work in **Internet of Things** for the academic year 2026–27, under the guidance of Prof. Gufran Qureshi. To the best of my knowledge, the work of the student is original and the information included in the practical is correct.

Internal Guide	**External Examiner**	**Head of the Department**

Date: _____ **College Seal:**

INDEX

Sr. No.	Title	Page No.
	MODULE 1	
1	Create different blinking patterns with LEDs by connecting to GPIO pins of Raspberry Pi.	1
2	Use Telegram to control Raspberry Pi GPIO.	3
3	Interface TM1637 with Raspberry Pi and Display current time.	6
4	Interface GPS module with Raspberry Pi.	8
5	Interface ADS 1115 with Raspberry Pi to act as oscilloscope.	10
6	Interface 16x2 LCD with Raspberry Pi using I2C module to display text.	12
7	Interface RFID module with Raspberry Pi to read and write to RFID tags/cards.	14
8	Interface Fingerprint Sensor with Raspberry Pi.	16
9	Interface Pi Camera with Raspberry Pi to capture image and video.	19
10	Design IoT-based home automation system using Raspberry Pi.	20
	MODULE 2	
11	Simulate door lock by interfacing fingerprint sensor and relay module with Raspberry Pi.	21
12	Implement basic Publisher – Subscriber using MQTT. Publish messages using paho mqtt. Subscribe and control LED.	23
13	Design a web dashboard using Flask. Control relay module via web.	25
14	Create a motion detector by interfacing camera and 16X2 display with Raspberry Pi. Print the appropriate message on the display.	26
15	Write a program to read the CPU%, Memory% and CPU temperature of Raspberry Pi.	27
16	Installing Windows 10 IoT Core on Raspberry Pi.	29

Internet of Things Practical

Module 1

Practical 1:

Aim: Create different blinking patterns with LEDs by connecting to GPIO pins of Raspberry Pi.

Connectors:

1. Connect the VGA to HDMI converter between the Raspberry Pi and monitor.
2. Connect the USB keyboard and mouse to the Raspberry Pi.
3. Connect the Micro USB 5V power supply to the Raspberry Pi.
4. Connect the Raspberry Pi to the breadboard using jumper wires.
5. Connect the cathode of the LED to the Ground (GND) pin.
6. Connect the anode of the LED to a resistor.

Code:

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)

# Set up GPIO pins

GPIO.setup(17, GPIO.OUT)
GPIO.setup(18, GPIO.OUT)
GPIO.setup(27, GPIO.OUT)
GPIO.setup(22, GPIO.OUT)

try:
    while True:

        # Pattern 1: All LEDs ON
        print("Ayesha, Saamiya, Mantasha, Harsh, Piyush")
        GPIO.output(17, GPIO.HIGH)
        GPIO.output(18, GPIO.HIGH)
        GPIO.output(27, GPIO.HIGH)
        GPIO.output(22, GPIO.HIGH)
        print("Pattern 1: All LEDs ON")
        time.sleep(0.4)

        # Pattern 2: All LEDs OFF
        GPIO.output(17, GPIO.LOW)
        GPIO.output(18, GPIO.LOW)
        GPIO.output(27, GPIO.LOW)
        GPIO.output(22, GPIO.LOW)
        print("Pattern 2: All LEDs OFF")
        time.sleep(0.4)

        # Pattern 3: LEDs 17 and 27 ON, 18 and 22 OFF
        GPIO.output(17, GPIO.HIGH)
        GPIO.output(18, GPIO.LOW)
        GPIO.output(27, GPIO.HIGH)
        GPIO.output(22, GPIO.LOW)
        print("Pattern 3: LEDs 17 and 27 ON")
        time.sleep(0.4)

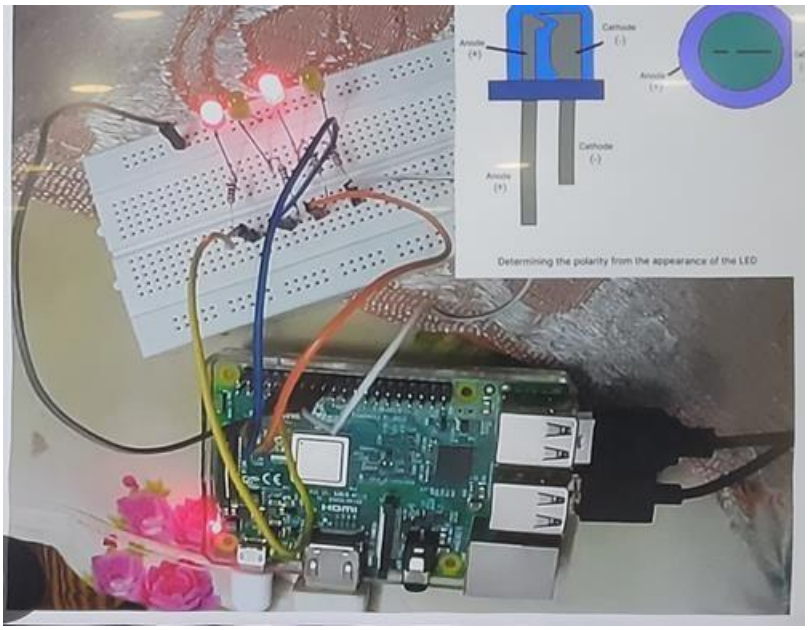
        # Pattern 4: LEDs 17 and 27 OFF, 18 and 22 ON
        GPIO.output(17, GPIO.LOW)
        GPIO.output(18, GPIO.HIGH)
        GPIO.output(27, GPIO.LOW)
        GPIO.output(22, GPIO.HIGH)
        print("Pattern 4: LEDs 18 and 22 ON")
        time.sleep(0.4)

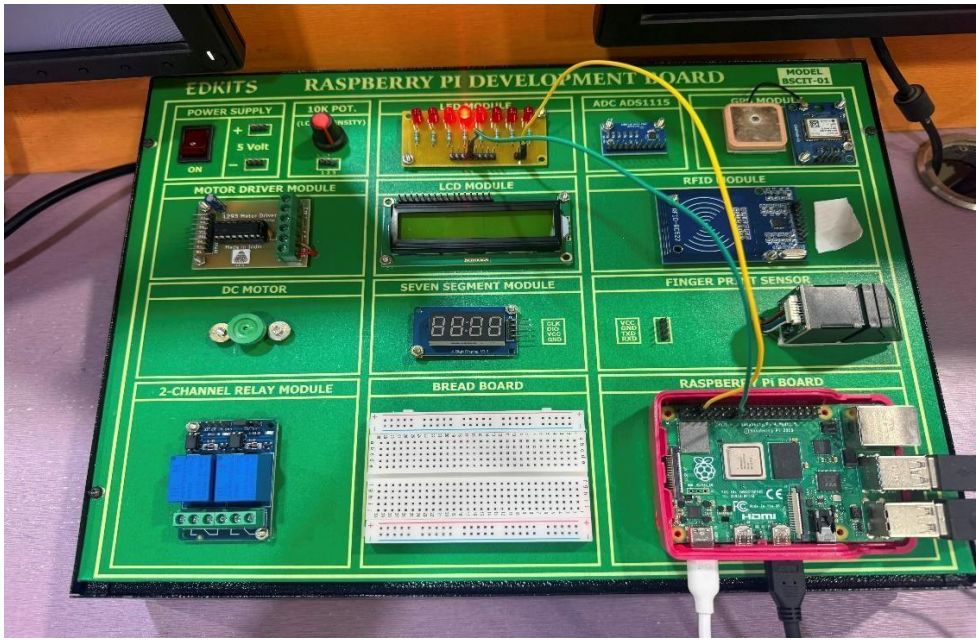
except KeyboardInterrupt:
```

```
    print("Program stopped")

finally:
    GPIO.cleanup()
```

Output:





```
Ayesha, Saamiya, Mantasha, Harsh, piyush
LED 17 & 22 ON
ALL LEDs ON
ALL LEDs OFF
LED 17 & 27 ON
LED 18 & 22 ON
ALL LEDs ON
ALL LEDs OFF
LED 17 & 27 ON
LED 18 & 22 ON
ALL LEDs ON
ALL LEDs OFF
LED 17 & 27 ON
LED 18 & 22 ON
```

Practical 2:

Aim: Use Telegram to control Raspberry Pi GPIO.

Connectors:

1. Connect the VGA to HDMI converter between the Raspberry Pi and monitor.
2. Connect the USB keyboard and mouse to the Raspberry Pi.
3. Connect the Micro USB 5V power supply to the Raspberry Pi.
4. Connect the Raspberry Pi to the breadboard using jumper wires.
5. Connect the cathode of the LED to the Ground (GND) pin.
6. Connect the anode of the LED to a resistor.

Code:

```
import sys
import time
import random
import datetime
import telepot
import RPi.GPIO as GPIO

GPIO.setwarnings(False)

# LED functions
def on(pin):
    GPIO.output(pin, GPIO.HIGH)
    return

def off(pin):
    GPIO.output(pin, GPIO.LOW)
    return

# Use Raspberry Pi BOARD pin numbering
GPIO.setmode(GPIO.BOARD)

# Set up GPIO output channel
GPIO.setup(11, GPIO.OUT)

# Telegram message handler
def handle(msg):
    chat_id = msg['chat']['id']
    command = msg['text']
    print("Ayesha, Saamiya, Mantasha, Harsh, Piyush")
    print("Got command: %s" % command)

    if command.lower() == 'on':
        on(11)
        bot.sendMessage(chat_id, "LED ON")

    elif command.lower() == 'off':
        off(11)
        bot.sendMessage(chat id, "LED OFF")

    else:
        bot.sendMessage(chat_id, "Send ON or OFF")

# Enter your Telegram bot token
bot = telepot.Bot('8892983249:AAGy0797GZh7DfWTJ3YQBhqCTE3BRY7gAQ')
bot.message_loop(handle)

print("Shadaan, Milan, Rahul, Shivam, Zaid, Ajay, Nitin")
print("I am listening...")

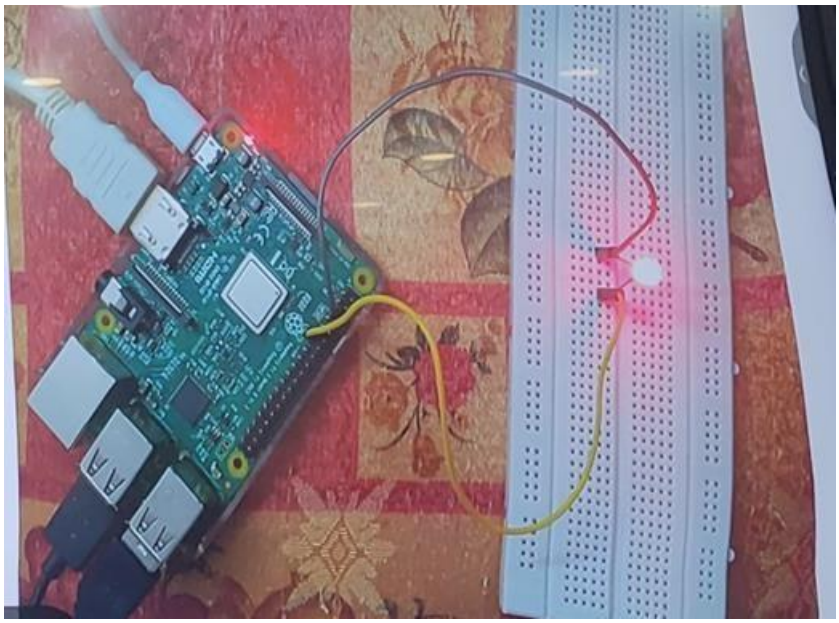
try:
    while 1:
        time.sleep(10)

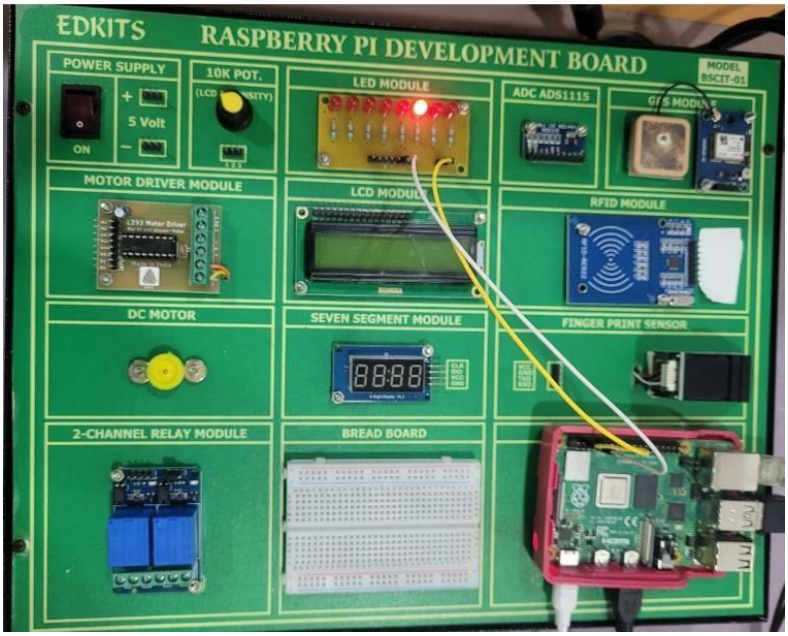
except KeyboardInterrupt:
```

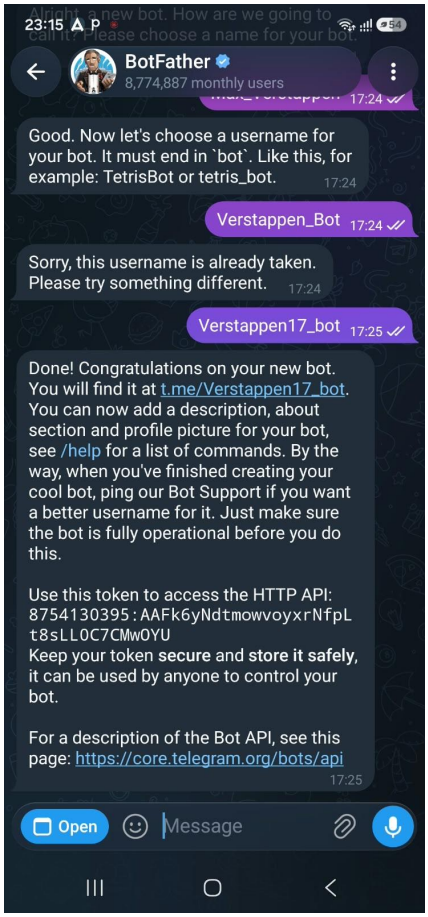
```
    print("Program stopped")

finally:
    GPIO.cleanup()
```

Output:










```
Ayesha, Saamiya, Mantasha, Harsh, Piyush
I am listening...

Ayesha, Saamiya, Mantasha, Harsh, Piyush
Got command: on

Ayesha, Saamiya, Mantasha, Harsh, Piyush
Got command: on

Ayesha, Saamiya, Mantasha, Harsh, Piyush
Got command: off
```

Practical 3:

Aim: Interface TM1637 with Raspberry Pi and Display current time.

Connectors:

1. Connect the VGA to HDMI converter between the Raspberry Pi and monitor.
2. Connect the USB keyboard and mouse to the Raspberry Pi.
3. Connect the Micro USB 5V power supply to the Raspberry Pi.
4. Connect the TM1637 to the Raspberry Pi using jumper wires.
    - CLK to 23rd pin (GPIO 11).
    - DIO to 24th pin (GPIO 08).
    - VCC to 2nd pin (5V).
    - GND to 6th pin (Ground).

Code:

```
import sys
import time
import datetime
import RPi.GPIO as GPIO
import tm1637

# Names
print("Ayesha, Saamiya, Mantasha, Harsh, Piyush")

# Initialize 7-segment display
Display = tm1637.TM1637(11, 8, tm1637.BRIGHT TYPICAL)

Display.Clear()
Display.SetBrightness(1)

try:
    while True:
        now = datetime.datetime.now()

        hour = now.hour
        minute = now.minute
        second = now.second

        currenttime = [
            int(hour / 10),
            hour % 10,
            int(minute / 10),
            minute % 10
        ]
        Display.Show(currenttime)
        Display.ShowDoublepoint(second % 2)

        print("Ayesha, Saamiya, Mantasha, Harsh, Piyush")
        print("Time:", now.strftime("%H:%M:%S"))

        time.sleep(1)

except KeyboardInterrupt:
    print("Program stopped")

finally:
    Display.Clear()
    GPIO.cleanup()
```

Output:




```
Ayesha, Saamiya, Mantasha, Harsh, Piyush
Time: 22:45:01
Time: 22:45:02
Time: 22:45:03
Time: 22:45:04
```

Practical 4:

Aim: Interface GPS module with Raspberry Pi.

Connectors:

1. Connect the VGA to HDMI converter with the Raspberry Pi.
2. Connect the USB keyboard and mouse with the Raspberry Pi.
3. Connect the Micro USB 5V power supply to the Raspberry Pi.
4. Use jumper wires to connect the GPS module with the Raspberry Pi.
    - VCC to Pin 2
    - RX to Pin 8 (TX)
    - TX to Pin 10 (RX)
    - GND to Pin 6
5. This module provides nonstop NMEA (National Marine Electronics Association) data strings to the TX pin, resulting in GPS information.

Code:

```
import serial
import time
import string
import pynmea2

print("Ayesha, Saamiya, Mantasha, Harsh, Piyush")
print("----------------------------------------")

port = "/dev/serial0"
ser = serial.Serial(port, baudrate=9600, timeout=0.5)

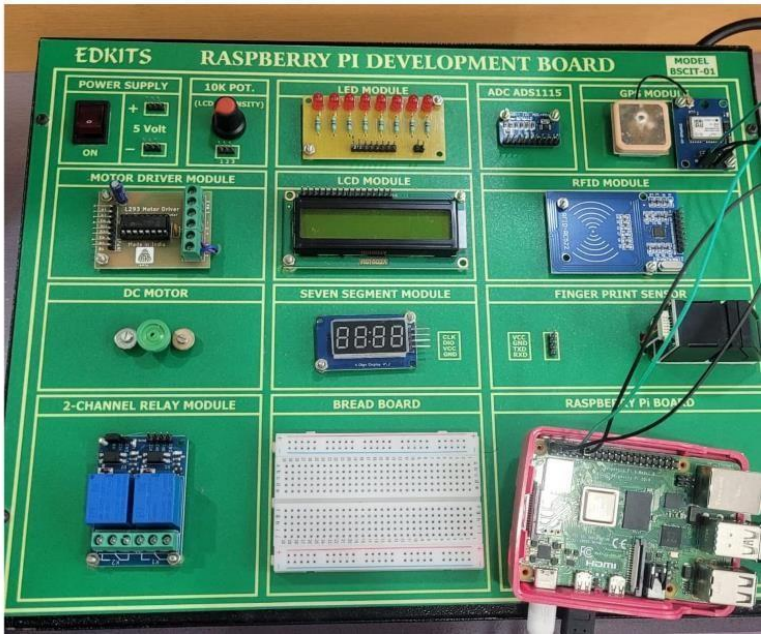
try:
    while True:
        newdata = ser.readline().decode(
            "ascii", errors="replace"
        ).strip()

        if newdata.startswith("$GPRMC"):
            newmsg = pynmea2.parse(newdata)
            lat = newmsg.latitude
            lng = newmsg.longitude
            gps = "Latitude=" + str(lat) + " and Longitude=" + str(lng)
            print(gps)

except KeyboardInterrupt:
    print("Program stopped")

finally:
    ser.close()
```

Output:




```
      print('Ayesha, Saamiya, Mantasha, Harsh, Piyush')

 SyntaxError: EOL while scanning string literal

>>> %Run gpss.py

  Ayesha, Saamiya, Mantasha, Harsh, Piyush
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
  Latitude=0.0 and Longitude=0.0
```

Practical 5:

Aim: Interface ADS 1115 with Raspberry Pi to act as oscilloscope.

Connectors:

1. Connect VGA to HDMI converter with Raspberry Pi & monitor.
2. Connect the USB keyboard and mouse with Raspberry Pi.
3. Connect the Micro USB 5V power supply to Raspberry Pi.
4. Connect ADC to Raspberry Pi using jumper wires.

    - VDD to 17th Pin (3.3 V)
    - GND to 9th Pin (GND)
    - SCL to 5th Pin (GPIO 03)
    - SDA to 3rd Pin (GPIO 02)

Code:

```
import time
import numpy.core.multiarray
import matplotlib.pyplot as plt
import Adafruit_ADS1x15
import RPi.GPIO as GPIO

adc = Adafruit ADS1x15.ADS1115()

GAIN = 16
val = []
cnt = 0

print("Ayesha, Saamiya, Mantasha, Harsh, Piyush")

plt.ion()
adc.start_adc(0, gain=GAIN)

print("Reading ADS1115 channel 0")

def makeFig():
    plt.ylim(-5000, 5000)
    plt.title("Oscilloscope")
    plt.grid(True)
    plt.ylabel("ADC outputs")
    plt.plot(val, 'r-o', label='lux')
    plt.legend(loc='lower right')

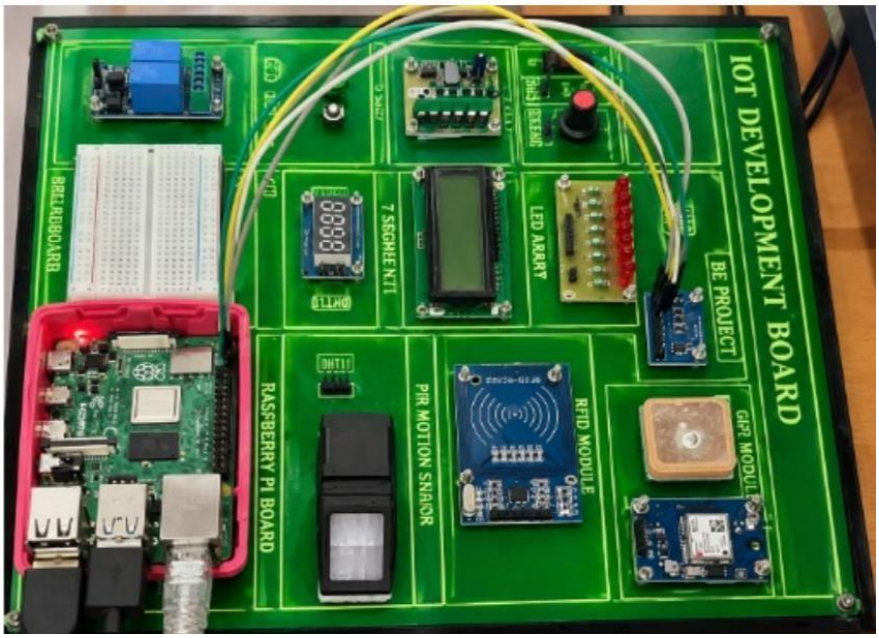
try:
    while True:
        value = adc.get_last_result()
        print("Channel0:{}".format(value))
        time.sleep(0.5)

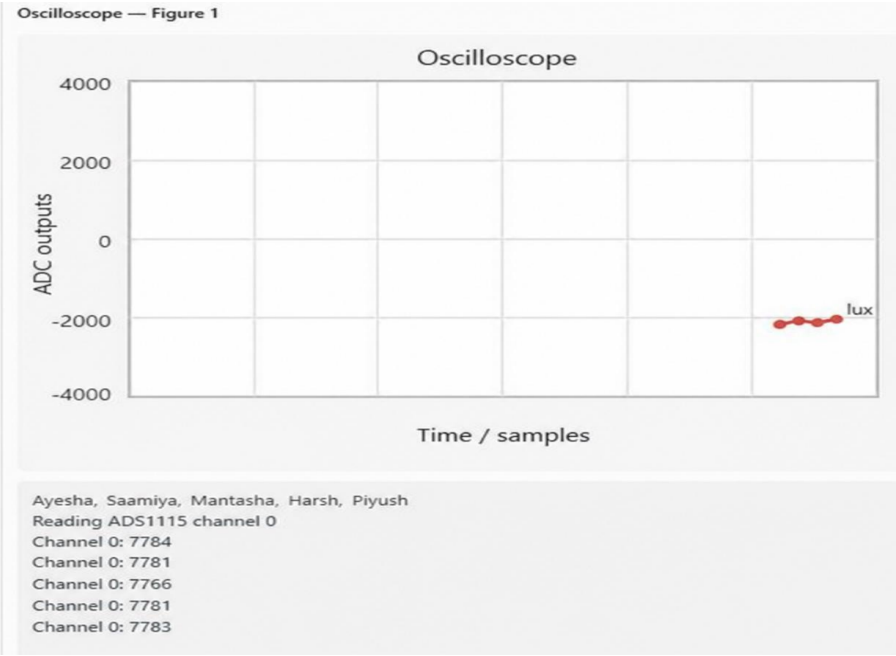
        val.append(int(value))
        plt.clf()
        makeFig()
        plt.pause(0.00001)

        cnt += 1
        if cnt > 50:
            val.pop(0)
            cnt = cnt - 1

except KeyboardInterrupt:
    print("Program stopped")
    plt.close()
```

Output:






```
Ayesha, Saamiya, Mantasha, Harsh, Piyush
Reading ADS1115 channel 0
Channel 0: 7784
Channel 0: 7781
Channel 0: 7766
Channel 0: 7781
Channel 0: 7783
```

Practical 6:

Aim: Interface 16x2 LCD with Raspberry Pi using I2C module to display text.

Code:

```
import time
import board
import digitalio
import adafruit character lcd.character lcd as characterlcd

lcd_columns = 16
lcd_lines = 2

lcd_rs = digitalio.DigitalInOut(board.D26)

lcd_en = digitalio.DigitalInOut(board.D19)

lcd_d4 = digitalio.DigitalInOut(board.D25)

lcd d5 = digitalio.DigitalInOut(board.D24)

lcd_d6 = digitalio.DigitalInOut(board.D22)

lcd_d7 = digitalio.DigitalInOut(board.D27)

lcd_backlight = digitalio.DigitalInOut(board.D4)

lcd = characterlcd.Character_LCD_Mono(

    lcd_rs, lcd_en, lcd_d4, lcd_d5, lcd_d6, lcd_d7,

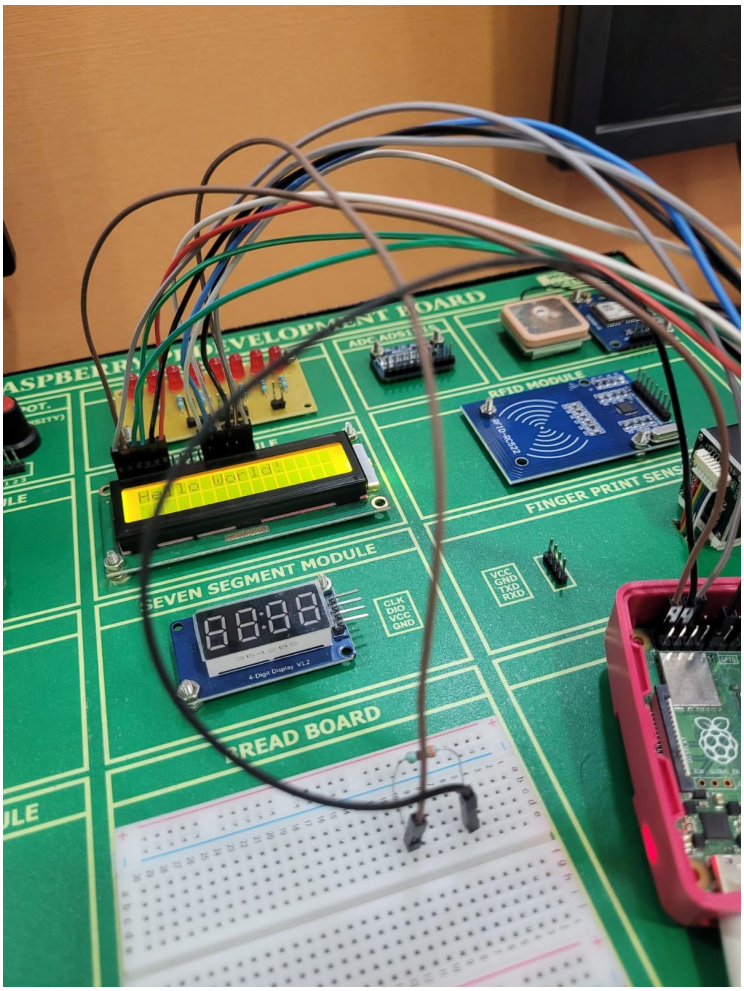
    lcd columns, lcd lines, lcd backlight

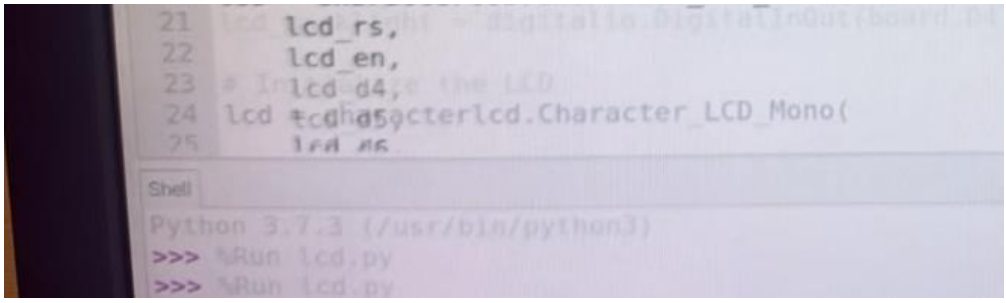
)

lcd.message = "Hello World"

time.sleep(5)
lcd.clear()
```

Output:


RFID MODULE
FINGER PRINT SENS
SEVEN SEGMENT MODULE
CLK
DIO
VCC
GND
READ BOARD


lcd_rs,
lcd_en,
lcd_d4,
lcd_d5,
Shell
Python 3.7.3 (/usr/bin/python3)
>>> %Run lcd.py
>>> %Run lcd.py

Practical 7:

Aim: Interface RFID module with Raspberry Pi to read and write to RFID tags/cards.

Write Code:

```
import RPi.GPIO as GPIO
from mfrc522 import SimpleMFRC522

rfid = SimpleMFRC522()

print("Ayesha, Saamiya, Mantasha, Harsh, Piyush")

try:
    print("Hold the RFID tag near the reader...")
    rfid.write("Milan")
    print("Data written successfully!")

except KeyboardInterrupt:
    print("Program stopped")

finally:
    GPIO.cleanup()
```

Read Code:

```
import RPi.GPIO as GPIO
from mfrc522 import SimpleMFRC522


rfid = SimpleMFRC522()

print("Ayesha, Saamiya, Mantasha, Harsh, Piyush")

try:
    while True:

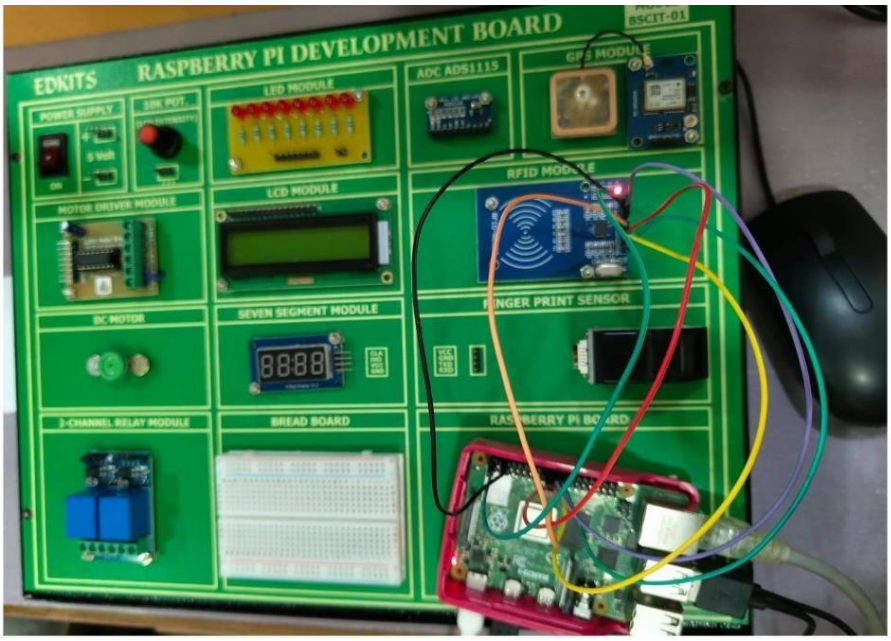
        id, text = rfid.read()

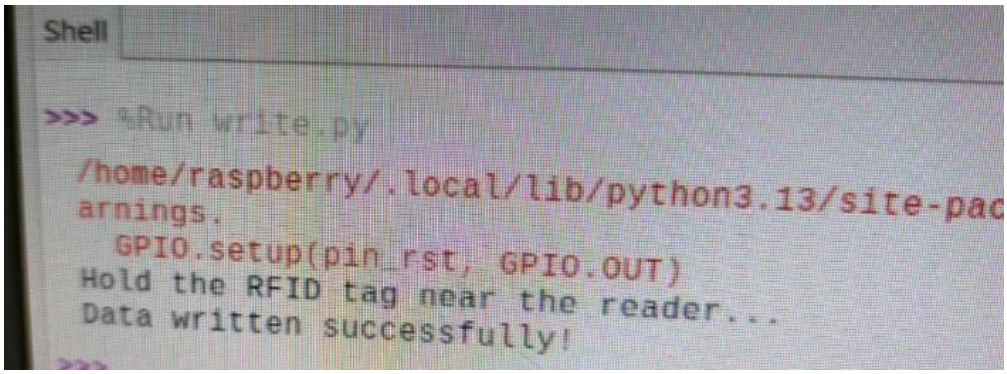
        print(id)
        print(text)
except KeyboardInterrupt:

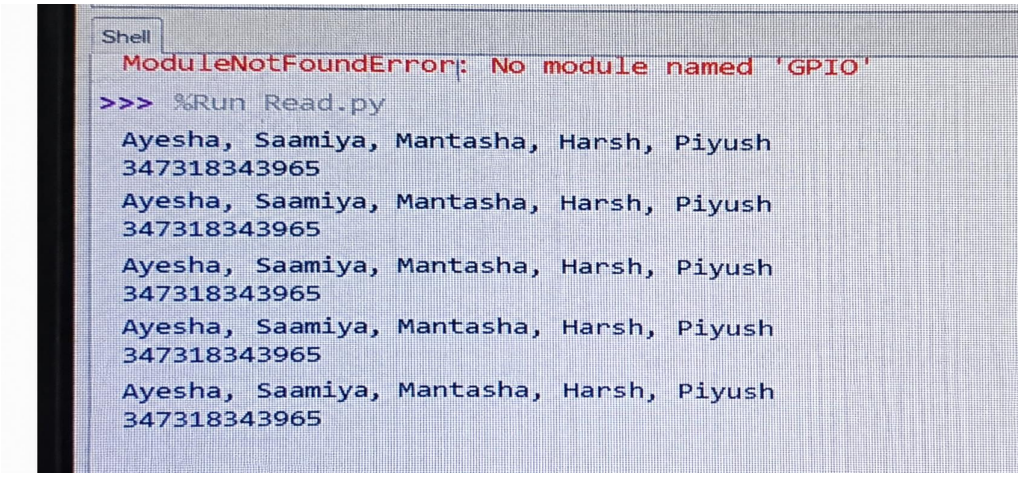
    print("Program stopped")


finally:
    GPIO.cleanup()
```

Output:


EDKITS
RASPBERRY PI DEVELOPMENT BOARD
LED MODULE
ADC ADS1115
GPS MODULE
LCD MODULE
RFID MODULE
SEVEN SEGMENT MODULE
BREAD BOARD


Shell
>>> %Run write.py
/home/raspberry/.local/lib/python3.13/site-pac
arnings.
GPIO.setup(pin_rst, GPIO.OUT)
Hold the RFID tag near the reader...
Data written successfully!


Shell
ModuleNotFoundError: No module named 'GPIO'
>>> %Run Read.py
Ayesha, Saamiya, Mantasha, Harsh, Piyush
347318343965
Ayesha, Saamiya, Mantasha, Harsh, Piyush
347318343965
Ayesha, Saamiya, Mantasha, Harsh, Piyush
347318343965
Ayesha, Saamiya, Mantasha, Harsh, Piyush
347318343965
Ayesha, Saamiya, Mantasha, Harsh, Piyush
347318343965

Practical 8:

Aim: Interface Fingerprint Sensor with Raspberry Pi.

Code:

```
import time
from pyfingerprint.pyfingerprint import PyFingerprint
import RPi.GPIO as gpio

print("Ayesha, Saamiya, Mantasha, Harsh, Piyush")
print("Successfully imported fingerprint module")

gpio.setwarnings(False)
gpio.setmode(gpio.BCM)

f = PyFingerprint('/dev/serial0', 57600, 0xFFFFFFFF, 0x00000000)


def enrollFinger():
    print('Enrolling Finger')
    time.sleep(3)
    print('Place Finger')

    while f.readImage() == False:
        pass

    f.convertImage(0x01)
    result = f.searchTemplate()
    positionNumber = result[0]

    if positionNumber >= 0:
        print('Template already exists at position #' + str(positionNumber))
        time.sleep(2)
        return

    print('Remove finger')
    print('Waiting for same finger')
    time.sleep(3)

    while f.readImage() == False:
        pass

    f.convertImage(0x02)

    if f.compareCharacteristics() == 0:
        print('Fingers do not match')
        time.sleep(2)
        return
    else:
        f.createTemplate()
        positionNumber = f.storeTemplate()
        print('Finger enrolled successfully')
        print('Stored at pos: ' + str(positionNumber))
        time.sleep(2)


def searchFinger():
    try:
        print('Waiting for finger...')
        time.sleep(2)

        while f.readImage() == False:
            pass

        f.convertImage(0x01)
        result = f.searchTemplate()
        positionNumber = result[0]

        if positionNumber == -1:
            print('No Match found')
            time.sleep(4)
            return False
        else:
            print('Found template at position ' + str(positionNumber))
            time.sleep(4)
            return True
```

```
    except Exception as e:
        print('Operation failed')
        print('Exception message: ' + str(e))
        return False


def deleteFinger():
    try:
        print('Waiting for finger...')
        time.sleep(2)

        while f.readImage() == False:
            pass

        time.sleep(1)
        f.convertImage(0x01)
        print('Start')

        result = f.searchTemplate()
        positionNumber = result[0]

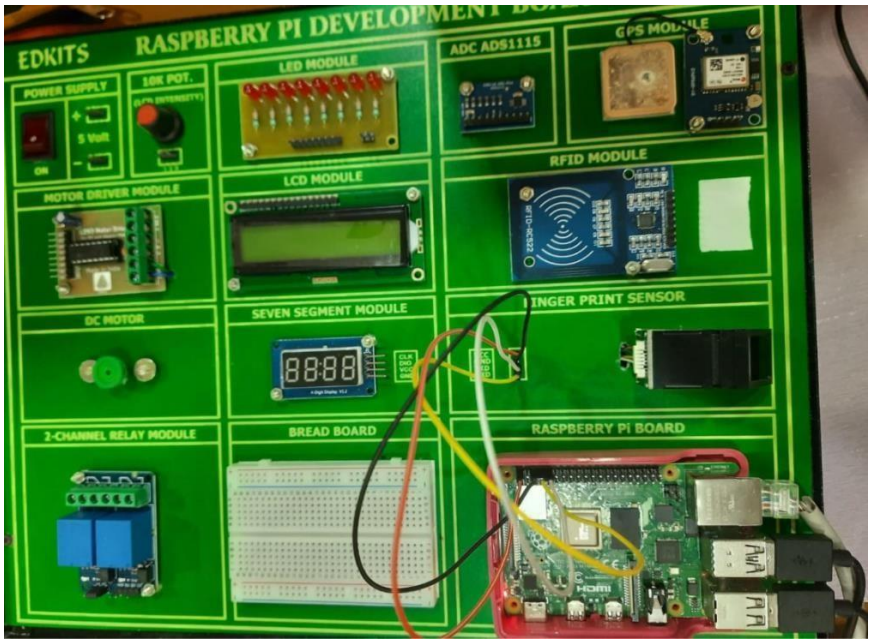
        if positionNumber == -1:
            print('No Match found')
            time.sleep(4)
            return False
        else:
            if f.deleteTemplate(positionNumber):
                print('Finger Deleted')
                time.sleep(4)
                return True

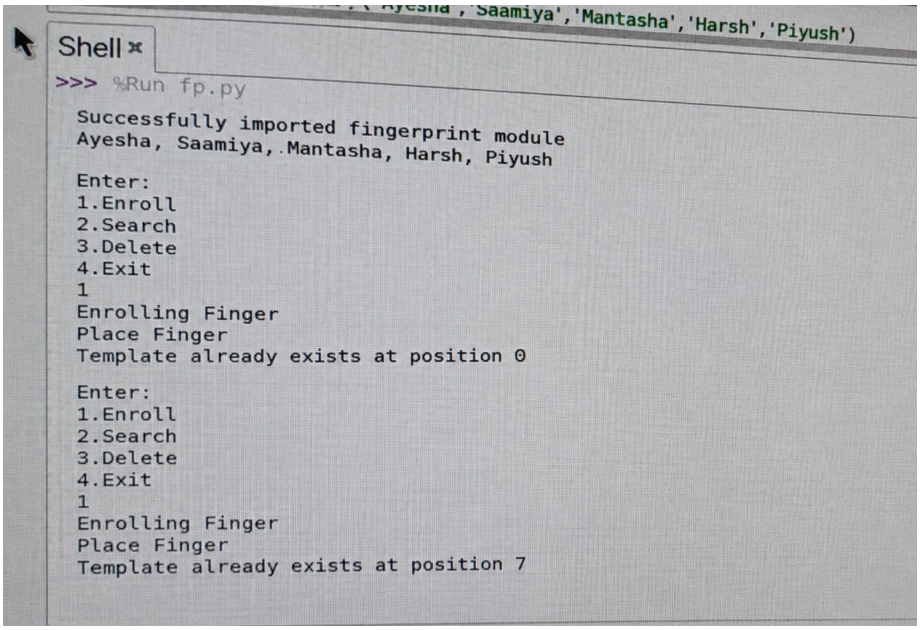
    except Exception as e:
        print('Operation failed')
        print('Exception message: ' + str(e))
        return False


while True:
    i = int(input("\nEnter:\n1.Enroll\n2.Search\n3.Delete\n4.Exit\n"))

    if i == 1:
        enrollFinger()
    elif i == 2:
        searchFinger()
    elif i == 3:
        deleteFinger()
    elif i == 4:
        break
    else:
        print("Invalid option!!! Enter Correct")
```

Output:


EDKITS
RASPBERRY PI DEVELOPMENT
POWER SUPPLY
5 Volt
10K POT.
LED MODULE
ADC ADS1115
RFID MODULE
MOTOR DRIVER MODULE
LCD MODULE
DC MOTOR
SEVEN SEGMENT MODULE
2-CHANNEL RELAY MODULE
BREAD BOARD
RASPBERRY PI BOARD


Saamiya','Mantasha','Harsh','Piyush')
Shell
>>> %Run fp.py
Successfully imported fingerprint module
Ayesha, Saamiya, Mantasha, Harsh, Piyush
Enter:
1.Enroll
2.Search
3.Delete
4.Exit
1
Enrolling Finger
Place Finger
Template already exists at position 0
Enter:
1.Enroll
2.Search
3.Delete
4.Exit
1
Enrolling Finger
Place Finger
Template already exists at position 7

Practical 9:

Aim: Interface Pi Camera with Raspberry Pi to capture image and video.

Code:

```
from picamera import PiCamera
from time import sleep

print("Ayesha, Saamiya, Mantasha, Harsh, Piyush")

camera = PiCamera()

camera.resolution = (1280, 780)

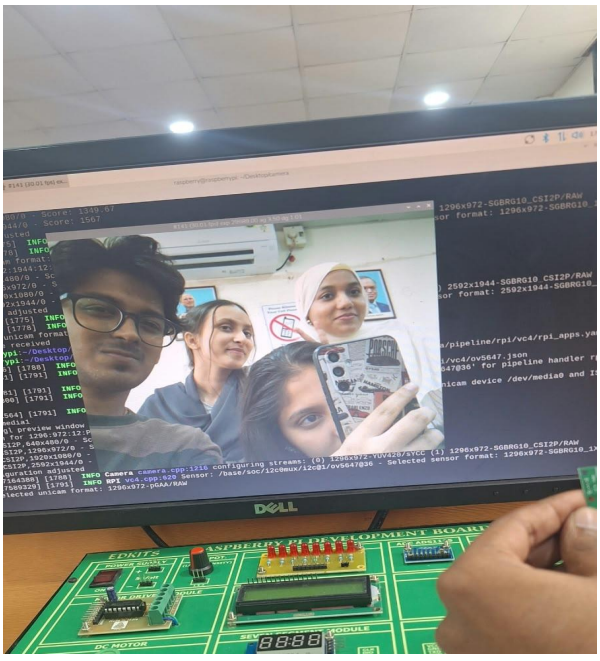
camera.start_preview()
camera.start_recording('video.h264')

for i in range(5):
    sleep(5)
    camera.capture('images%s.jpg' % i)

camera.stop_recording()
camera.stop_preview()
camera.close()
```

Output:





Practical 10:

Aim: Design IoT-based home automation system using Raspberry Pi.

Code:

```
sudo apt-get update
sudo apt-get upgrade
sudo reboot

cd ~

wget http://sourceforge.net/projects/webiopi/files/WebIOPi-0.7.1.tar.gz

tar xvzf WebIOPi-0.7.1.tar.gz

cd WebIOPi-0.7.1/

wget https://raw.githubusercontent.com/doublebind/raspi/master/webiopi-pi2bplus.patch

patch -p1 -i webiopi-pi2bplus.patch

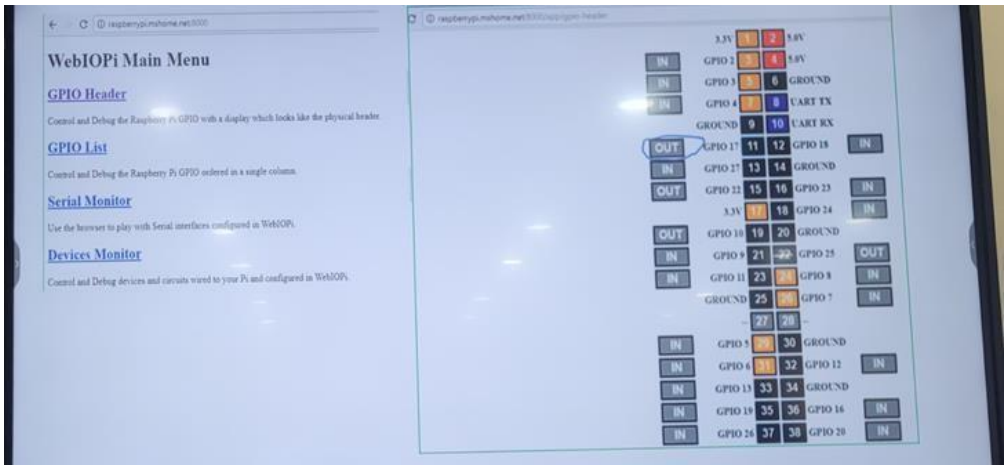
sudo chmod 775 setup.sh

sudo ./setup.sh

sudo reboot

sudo webiopi -d -c /etc/webiopi/config
```

Output:



Module 2

Practical 12:

Aim: Simulate door lock by interfacing fingerprint sensor and relay module with Raspberry Pi.

Components Required:

- Raspberry Pi Model B+
- 5V Micro USB Power Supply
- Monitor
- USB Keyboard and Mouse
- Fingerprint sensor module (UART-compatible, such as R305 or AS608)
- 1-channel relay module
- Jumper wires
- Breadboard, if required
- LED or other low-voltage load to simulate the door lock

Theory: A fingerprint sensor is a biometric device that captures a person's fingerprint and compares it with previously enrolled fingerprints. In this practical, the fingerprint sensor communicates with the Raspberry Pi through a UART serial connection. When a fingerprint is scanned, the program checks whether it matches a fingerprint stored in the sensor's memory. If an authorized fingerprint is identified, the Raspberry Pi activates the relay to simulate unlocking the door. If the fingerprint is not recognized, the relay remains inactive and the door stays locked. The relay acts as an electrically controlled switch. An LED or another suitable low-voltage load can be used to demonstrate the locking and unlocking process.

Code:

```
import time
import serial
import RPi.GPIO as GPIO
import adafruit_fingerprint

print("Ayesha, Saamiya, Mantasha, Harsh, Piyush")
print("Fingerprint Door Lock System")
print("Place your finger on the sensor")

# Relay configuration
RELAY = 11

GPIO.setmode(GPIO.BOARD)
GPIO.setwarnings(False)
GPIO.setup(RELAY, GPIO.OUT)
GPIO.output(RELAY, GPIO.LOW)

try:
    # Connect to the UART fingerprint sensor
    uart = serial.Serial(
        "/dev/serial0",
        baudrate=57600,
        timeout=1
    )

    finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)

    while True:
        result = finger.get_image()

        if result == adafruit_fingerprint.OK:
```

```
            # Convert the captured image
            result = finger.image_2_tz(1)

            if result == adafruit_fingerprint.OK:
                # Search enrolled fingerprints
                result = finger.finger_search()

                if result == adafruit fingerprint.OK:
                    print("Access Granted")
                    print("Fingerprint ID:", finger.finger_id)

                    # Unlock the door
                    GPIO.output(RELAY, GPIO.HIGH)
                    print("Door Unlocked")

                    time.sleep(5)

                    # Lock the door again
                    GPIO.output(RELAY, GPIO.LOW)
                    print("Door Locked")

                elif result == adafruit_fingerprint.NOTFOUND:
                    print("Access Denied")
                    print("Unauthorized Fingerprint")
                else:
                    print("Fingerprint search failed")

        elif result != adafruit fingerprint.NOFINGER:
            print("Unable to capture fingerprint")

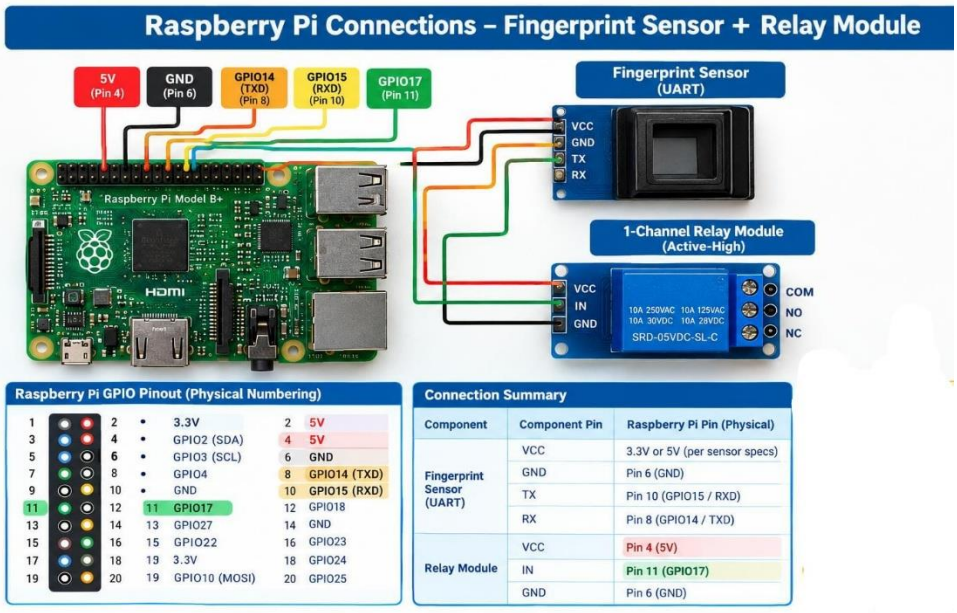
        time.sleep(0.2)

except KeyboardInterrupt:
    print("Program stopped")

except Exception as error:
    print("Error:", error)

finally:
    GPIO.output(RELAY, GPIO.LOW)
    GPIO.cleanup()
```

Output:



Practical 13:

Aim: Implement basic Publisher – Subscriber using MQTT. Publish messages using paho mqtt. Subscribe and control LED.

Components Required: Raspberry Pi Model B+, 5V Micro USB Power Supply, Monitor, USB Keyboard & Mouse, LED, 220Ω Resistor, Jumper Wires, Internet Connection.

Theory: MQTT (Message Queuing Telemetry Transport) is a lightweight communication protocol commonly used in IoT applications. It follows the Publisher-Subscriber model. The Publisher sends messages to a topic, while the Subscriber receives messages from that topic through an MQTT Broker. In this practical, one Python program publishes ON or OFF messages using the paho-mqtt library, and another Python program subscribes to the same topic and controls an LED.

Publisher Code (publisher.py)

```
import paho.mqtt.client as mqtt

broker = "broker.hivemq.com"
topic = "raspberrypi/led"

client = mqtt.Client()
client.connect(broker, 1883, 60)

while True:
    message = input("Enter ON or OFF: ")
    client.publish(topic, message)
    print("Message Published:", message)

```

Subscriber Code (subscriber.py)

```
import paho.mqtt.client as mqtt
import RPi.GPIO as GPIO

LED = 11

GPIO.setmode(GPIO.BOARD)
GPIO.setup(LED, GPIO.OUT)

def message_received(client, userdata, message):
    data = message.payload.decode()
    print("Message Received:", data)

    if data == "ON":
        GPIO.output(LED, GPIO.HIGH)
    elif data == "OFF":
        GPIO.output(LED, GPIO.LOW)

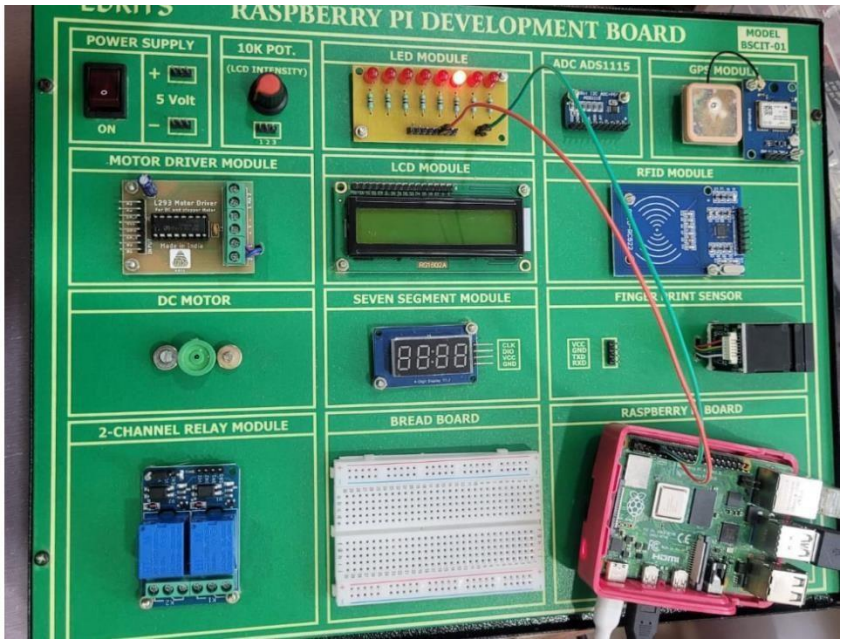
client = mqtt.Client()
client.on_message = message_received

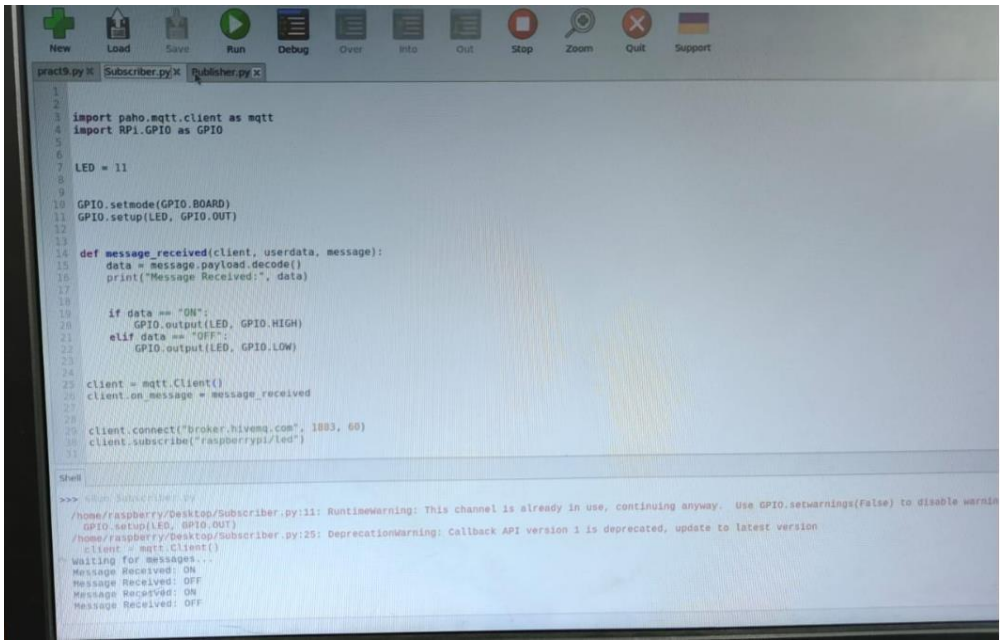
client.connect("broker.hivemq.com", 1883, 60)
client.subscribe("raspberrypi/led")

print("Waiting for messages...")
client.loop_forever()

```

Output:


RASPBERRY PI DEVELOPMENT BOARD
MODEL BSCIT-01
POWER SUPPLY
ON
5 Volt
10K POT.
(LCD INTENSITY)
LED MODULE
ADC ADS1115
MOTOR DRIVER MODULE
LCD MODULE
RFID MODULE
DC MOTOR
SEVEN SEGMENT MODULE
2-CHANNEL RELAY MODULE
BREAD BOARD


New
Load
Save
Run
Debug
Over
Into
Out
Stop
Zoom
Quit
Support
pract9.py
Subscriber.py
Publisher.py
import paho.mqtt.client as mqtt
import RPi.GPIO as GPIO
LED = 11
GPIO.setmode(GPIO.BOARD)
GPIO.setup(LED, GPIO.OUT)
def message_received(client, userdata, message):
data = message.payload.decode()
print("Message Received:", data)
if data == "ON":
GPIO.output(LED, GPIO.HIGH)
elif data == "OFF":
GPIO.output(LED, GPIO.LOW)
client = mqtt.Client()
client.on_message = message_received
client.connect("broker.hivemq.com", 1883, 60)
client.subscribe("raspberrypi/led")
Shell
/home/raspberry/Desktop/Subscriber.py:11: RuntimeWarning: This channel is already in use, continuing anyway. Use GPIO.setwarnings(False) to disable warnin
GPIO.setup(LED, GPIO.OUT)
/home/raspberry/Desktop/Subscriber.py:25: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
client = mqtt.Client()
Waiting for messages...
Message Received: ON
Message Received: OFF
Message Received: ON
Message Received: OFF

Practical 14:

Aim: Design a web dashboard using Flask. Control relay module via web.

Components Required: Raspberry Pi Model B+, 5V Micro USB Power Supply, Monitor, USB Keyboard & Mouse, 1-Channel Relay Module, Jumper Wires, Electrical Load/LED.

Theory: Flask is a lightweight Python web framework used to create web applications. In this practical, Flask is used to create a simple web dashboard with ON and OFF buttons. The Raspberry Pi receives the command from the web page and controls a relay module through a GPIO pin.

App.py

```
import paho.mqtt.client as mqtt
import RPi.GPIO as GPIO

LED = 11

GPIO.setmode(GPIO.BOARD)
GPIO.setup(LED, GPIO.OUT)

def message_received(client, userdata, message):
    data = message.payload.decode()
    print("Message Received:", data)

    if data == "ON":
        GPIO.output(LED, GPIO.HIGH)
    elif data == "OFF":
        GPIO.output(LED, GPIO.LOW)

client = mqtt.Client()
client.on_message = message_received

client.connect("broker.hivemq.com", 1883, 60)
client.subscribe("raspberrypi/led")

print("Waiting for messages...")
client.loop_forever()

```

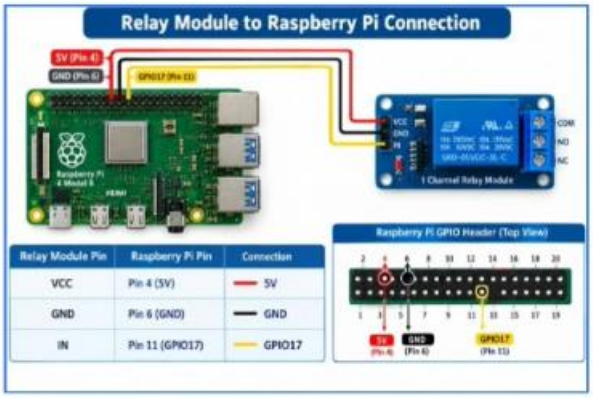
Output:

module via web

Required: Raspberry Pi Model B+, 5V Micro USB Power Supply, Monitor, USB Keyboard & Mouse, 1-
/ Module, Jumper Wires, Electrical Load/LED.
‹ is a lightweight Python web framework used to create web applications. In this practical, Flask is used to
le web dashboard with ON and OFF buttons. The Raspberry Pi receives the command from the web page
relay module through a GPIO pin.

y module to Raspberry Pi:
4 (5V)
6 (GND)
(GPIO17)

--break-system-packages



Practical 16:

Aim: Create a motion detector by interfacing camera and 16X2 display with Raspberry Pi. Print the appropriate message on the display.

Components Required: Raspberry Pi Model B+

- 5V Micro USB Power Supply
- Monitor
- USB Keyboard and Mouse
- Raspberry Pi-compatible USB camera or Raspberry Pi camera module
- 16×2 LCD display
- I²C LCD interface module (PCF8574), if available
- Jumper wires
- Breadboard, if required

Theory: Motion detection is a technique used to identify movement in a particular area. In this practical, the camera captures successive images, and Python compares them to identify changes between frames. If the difference exceeds a predefined threshold, motion is detected.

```
import cv2
import time
import RPi.GPIO as GPIO
from LCD import lcd
from CheckLCD import CheckCD

lcd = CheckCD()
PORTBY5F4R
address = 0x27
port = 1
eole = 16
rows = 2

camera = None

try:
    # Initialize the USB camera
    camera = cv2.VideoCapture(0)

    if not camera.isOpened():
        lcd.clear()
        lcd.write_string("Camera Error")
        print("Unable to open camera.")
        raise RuntimeError("Camera not detected")

    lcd.clear()
    lcd.write_string("Motion Detector")
    lcd.cursor_pos = (1, 0)
    lcd.write_string("No Motion")
    time.sleep(2)

    # Capture the initial frame
    ret, frame1 = camera.read()
    if not ret:
        raise RuntimeError("Unable to capture frame")

    gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
    gray1 = cv2.GaussianBlur(gray1, (21, 21), 0)

    while True:
        ret, frame2 = camera.read()
        if not ret:
            print("Camera frame unavailable")
            break

        # Convert the current frame to grayscale
        gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)
```

```
        gray2 = cv2.GaussianBlur(gray2, (21, 21), 0)

        # Compare consecutive frames
        difference = cv2.absdiff(gray1, gray2)
        _, threshold = cv2.threshold(
            difference, 25, 255, cv2.THRESH_BINARY
        )

        # Count changed pixels
        changed_pixels = cv2.countNonZero(threshold)
        lcd.clear()

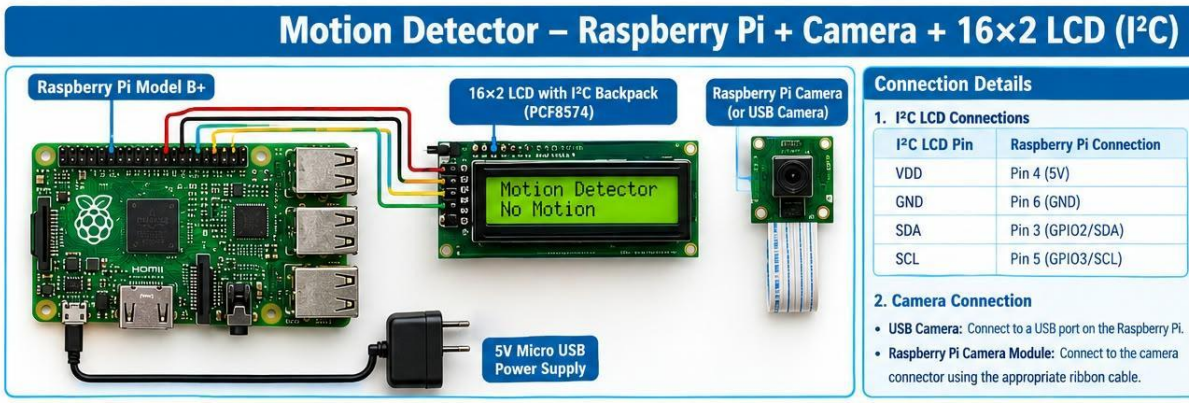
        if changed_pixels > 1500:
            lcd.write_string("Motion Detected!")
            print("Motion Detected!")
        else:
            lcd.write_string("No Motion")
            print("No Motion")

        # Use the current frame for the next comparison
        gray1 = gray2

        # Press q to stop
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

except KeyboardInterrupt:
    print("Program stopped")
except Exception as error:
    print("Error:", error)
finally:
    if camera is not None:
        camera.release()
    cv2.destroyAllWindows()
    lcd.clear()
    lcd.write_string("System Stopped")
    time.sleep(2)
    lcd.clear()
```

Output:



Practical 18:

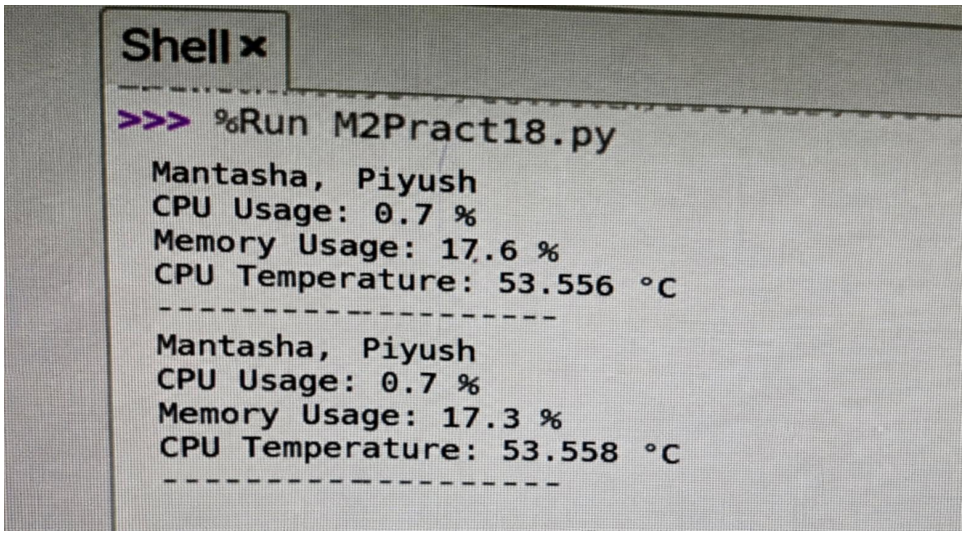
Aim: Write a program to read the CPU%, Memory% and CPU temperature of Raspberry Pi.

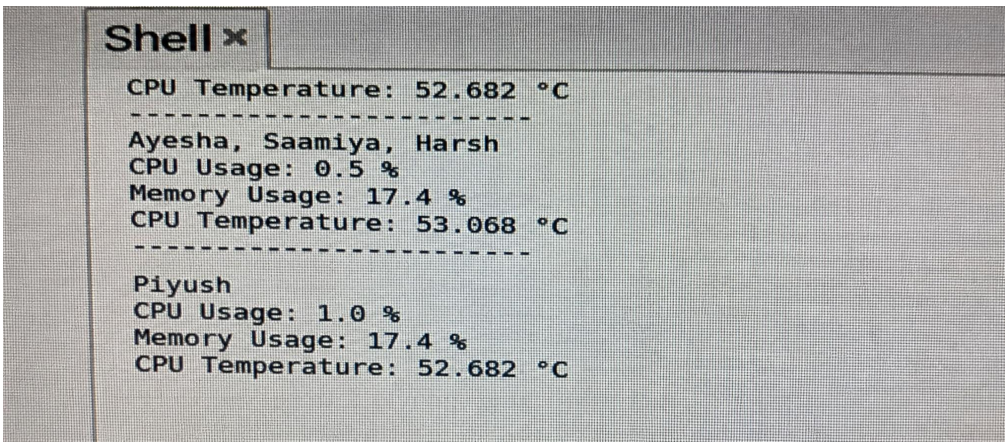
Components Required: Raspberry Pi Model B+, 5V Micro USB Power Supply, VGA to HDMI Converter, Monitor, USB Keyboard & Mouse, Raspberry Pi OS, Internet Connection.

Theory: The Raspberry Pi continuously uses its CPU and memory to run different programs and processes. CPU utilization indicates the percentage of CPU resources currently being used, while memory utilization indicates the percentage of RAM being used. The CPU temperature represents the temperature of the Raspberry Pi's processor. Monitoring these parameters is useful for checking the performance and health of the Raspberry Pi. Python provides libraries such as psutil to obtain CPU and memory usage. The CPU temperature can be obtained from the Raspberry Pi's system temperature file.

```
1. import psutil
2. import time
3.
4. print("Ayesha, Saamiya, Mantasha, Harsh, Piyush")
5.
6. while True:
7.     cpu = psutil.cpu_percent(interval=1)
8.     memory = psutil.virtual_memory().percent
9.
10.    with open("/sys/class/thermal/thermal_zone0/temp", "r") as file:
11.        temp = int(file.read()) / 1000
12.
13.    print("CPU Usage:", cpu, "%")
14.    print("Memory Usage:", memory, "%")
15.    print("CPU Temperature:", temp, "°C")
16.    print("----------------------")
17.
18.    time.sleep(2)
19.
```

Output:




Shell
CPU Temperature: 52.682 °C
Ayesha, Saamiya, Harsh
CPU Usage: 0.5 %
Memory Usage: 17.4 %
CPU Temperature: 53.068 °C
Piyush
CPU Usage: 1.0 %
Memory Usage: 17.4 %
CPU Temperature: 52.682 °C

Practical 20:

Aim: Installing Windows 10 IoT Core on Raspberry Pi.

Practical 20: Installing Windows 10 IoT Core on Raspberry Pi

- Go to the official website of Microsoft which is https://docs.microsoft.com/en-us/windows/iot-core/downloads.
- Click on the option "Download the Windows 10 IoT Core Dashboard".
- After downloading the above image, click the "Install" option.
- After installation click on "Run" option.
- IoT Dashboard window will appear. Click on "Set up a new device" option.
- A window will appear as shown below. Insert the SD card & select the drive. Set the root password and check the software license terms checkbox. Finally click on "Download & Install" option.











- A pop-up window will appear to erase the SD Card. Click on "Continue" option to erase existing data if any.
- This will start downloading Windows 10 IoT Core. Once downloaded it will start with flashing the SD card/Device.
- Lastly a popup menu will appear to format or cancel the SD card. Simply click on "Cancel Option".



